REPUBLIC OF TÜRKİYE

FATIH SULTAN MEHMET VAKIF UNIVERSITY

Faculty of Engineering

Department of Computer Engineering

UNDERGRADUATE GRADUATION PROJECT

INTERACTIVE AI-BASED SOFTWARE DEVELOPMENT PLATFORM WITH LLM AGENTS

Ömer Yasir ÖNAL – 2221221562

Ayşe Serra GÜMÜŞTAKIM – 2121251008

Project Advisor: Dr. Nazlı DOĞAN

Istanbul, January 2026

REPUBLIC OF TÜRKİYE

FATIH SULTAN MEHMET VAKIF UNIVERSITY

Faculty of Engineering

Department of Computer Engineering

UNDERGRADUATE GRADUATION PROJECT

INTERACTIVE AI-BASED SOFTWARE DEVELOPMENT PLATFORM WITH LLM AGENTS

Ömer Yasir ÖNAL – 2221221562

Ayşe Serra GÜMÜŞTAKIM – 2121251008

Project Advisor: Dr. Nazlı DOĞAN

Jury Members:	**Signature:**
……	……
……	……
……	……

Istanbul, January 2026

Table of Contents

PREFACE 5
ABSTRACT 6
SUMMARY 7
LIST OF FIGURES 8
LIST OF TABLES 9
3.4. CLOUD AND INFRASTRUCTURE REQUIREMENTS .....ERROR! BOOKMARK NOT DEFINED.
3.5. ACADEMIC REQUIREMENTS ..... ERROR! BOOKMARK NOT DEFINED.

1. INTRODUCTION 10
   1.1. PROBLEM DESCRIPTION AND MOTIVATION 10
   1.2. PROJECT SCOPE 10
   1.3. TERMS AND ABBREVIATIONS 11
2. RELATED WORK / LITERATURE SURVEY 12
   2.1. RESEARCH METHODOLOGY 12
   2.2. SUMMARY OF FINDINGS AND RELATION TO AKIS 12
3. PROJECT REQUIREMENTS 13
   3.1. FUNCTIONAL REQUIREMENTS 13
   3.2. NON-FUNCTIONAL REQUIREMENTS 14
   3.3. DESIGN CONSTRAINTS AND ASSUMPTIONS 14
4. SYSTEM DESIGN 15
   4.1. OVERALL ARCHITECTURE OVERVIEW 15
   4.2. BACKEND DESIGN 16
   4.2.1. AgentOrchestrator and Agent Lifecycle

..... 16
4.2.2. Authentication and Onboarding Flows
..... 17
4.2.3. API Layer and Endpoint Summary
..... 17
4.2.4. Security Layer and Middleware
..... 18
4.2.5. Module Structure and File Organization
..... 18
4.3. FRONTEND DESIGN
..... 18
4.3.1. Information Architecture and Screens
..... 18
4.4. DATA MODEL DESIGN
..... 18
4.4.1. Entity and Relational Model Overview
..... 19
4.4.2. Job, Trace Log, and Artifact Lifecycle
..... 19
4.4.3. Table Fields (Column-Level)
..... 20
4.5. INTEGRATION DESIGN
..... 29
4.5.1. MCP Gateway and Adapter Approach
..... 29
4.5.2. Docker Components and Execution Model
..... 30
4.5.3. Integration Security and Access Controls
..... 30
4.6. TEST PLAN AND QUALITY STRATEGY
..... 30
4.7. DEPLOYMENT AND OPERATIONS
..... 31
5. TASKS ACCOMPLISHED
..... 31
5.1. COMPLETED WORK AND INTERMEDIATE DELIVERABLES
..... 31
5.2. CURRENT STATUS TABLE
..... 32
REFERENCES
..... 32

PREFACE

This report presents the solution design and technical analysis of the **AKIS Platform**, developed as part of the Undergraduate Graduation Project of the Computer Engineering Department at Fatih Sultan Mehmet Vakıf University. The primary aim of this work is to deliver a prototype that improves efficiency and auditability in high-effort stages of the software development life cycle (SDLC)—such as documentation generation, trace-based review, and external system integrations—by leveraging an **LLM-based agent approach**. Within this scope, AKIS targets an end-to-end execution framework built on centralized orchestration, secure integration adapters, and persistent trace/artifact management.

We would like to express our sincere gratitude to our project advisor, **Assist. Prof. Dr. Nazlı DOĞAN**, for her guidance and valuable contributions throughout this study. We also thank the team members for their collaboration and efforts during the development process.

Sincerely,

Ömer Yasir Önal

04.01.2026

AKIS Platform: An Interactive Software Development and Governance System with LLM-Based Agents

ABSTRACT

This document presents the solution design and technical analysis of the AKIS Platform. AKIS is an interactive software platform prototype that aims to reduce repetitive and low-value activities in the Software Development Life Cycle (SDLC) through LLM-based agents. The platform is designed as a modular monolith backend (Fastify + TypeScript) with a decoupled SPA frontend (React + Vite), and persists job execution results, traces, and produced artifacts in PostgreSQL via Drizzle.

At the current stage, the orchestration core (AgentOrchestrator), the job lifecycle state machine, multi-step sign-up/login and email verification flows, and a dashboard experience to configure agents and run demo scenarios have been implemented. The design is shaped around auditability (trace and artifact persistence), security controls (rate limiting, secure session cookies), low dependencies, and resource-efficient operation in constrained cloud environments.

SUMMARY

AKIS is positioned as a platform-level implementation of agentic automation for software delivery workflows. Instead of building single-purpose tools, AKIS unifies documentation generation (AKIS Scribe), test automation assistance (AKIS Trace), and requirements-to-prototype flows (AKIS Proto) under a single orchestration kernel and a standardized integration layer based on the Model Context Protocol (MCP).

The platform adopts a contract-oriented architecture with explicit job states and persistent evidence (plans, traces, artifacts). This enables repeatable execution, post-hoc review, and controlled operation in scenarios that may require human approval. The deployment target is Oracle Cloud Infrastructure (OCI) Free Tier; therefore, the solution prioritizes simplicity, minimal runtime overhead, and limited operational complexity.

LIST OF FIGURES

Figure 4.1: AKIS Platform overall architecture components (backend–frontend–database–MCP integrations).

Figure 4.2: Finite state machine (FSM) summary for the job lifecycle.

LIST OF TABLES

Table 3.1: Summary of Functional Requirements (FR).

Table 3.2: Summary of Non-Functional Requirements (NFR).

Table 4.1: Database tables and their purposes (summary).

Table 4.2: jobs table fields (selected columns).

Table 4.3: job_traces table fields (selected columns).

Table 5.1: Current status table (phases and focus areas).

1. INTRODUCTION

1.1. Problem Definition and Motivation

Modern software development processes (Software Development Life Cycle—SDLC) create significant operational load for teams due to growing codebase complexity, frequently changing requirements, and accelerating delivery expectations. This project targets three problem areas that reduce productivity and increase costs: (i) outdated documentation, (ii) high test automation cost, and (iii) slow MVP prototyping.

The AKIS Platform aims to automate repetitive, low-value SDLC tasks through LLM-based agents, enabling teams to focus on higher-value engineering decisions and product development. The platform unifies documentation updates (AKIS Scribe), test automation generation (AKIS Trace), and requirements-driven prototyping (AKIS Proto) under a single orchestration core.

1.2. Project Scope

AKIS is designed with a modular monolith backend within a single repository and a decoupled SPA frontend. The system consists of a central orchestrator (AgentOrchestrator) that manages the agent execution lifecycle, agent implementations, and an MCP adapter layer that connects to external systems. The deployment target is a prototype that can run with low dependencies on Oracle Cloud Infrastructure (OCI) Free Tier resources.

Key in-scope components are summarized below:

- Platform core: job creation and monitoring, state management, trace/log persistence, and artifact storage.
- Agents: minimum viable functionality and configuration parameters for AKIS Scribe, AKIS Trace, and AKIS Proto.
- Integrations: access to systems such as GitHub/Jira/Confluence through Model Context Protocol (MCP) adapters.
- Authentication and onboarding: email-based multi-step sign-up/login flows and stateless JWT sessions.
- Academic deliverables: architecture explanations and the project report set.

Out-of-scope items include:

- No-code/low-code agent creation wizards or user-programmable agent logic.

- Enterprise-scale team management features such as multi-layer RBAC and on-premise deployment options.
- Production-grade deployment with Kubernetes and comprehensive enterprise CI/CD automation.
- Billing/invoicing modules and a multi-tier subscription infrastructure.

1.3. Terms and Abbreviations

Term	Definition
AKIS	Interactive software development platform with LLM-based agents.
LLM	Large Language Model.
MCP	Model Context Protocol; standardized integration layer for external systems.
OCI	Oracle Cloud Infrastructure.
JWT	JSON Web Token; stateless session token stored in an HTTP-only cookie.
SDLC	Software Development Life Cycle.

2. RELATED WORK / LITERATURE SURVEY

2.1. Research Methodology

This study surveys recent research on the use of LLM-based autonomous agents in software engineering processes, including automated documentation generation, test generation, and verification mechanisms. The survey focuses on peer-reviewed work published between 2020 and 2025, using sources such as IEEE Xplore, ACM Digital Library, SpringerLink, and Google Scholar. Findings are synthesized and grouped under autonomous agent architectures, documentation generation, test generation, and reasoning/acting strategies.

2.2. Summary of Findings and Relation to AKIS

Prior work demonstrates that LLM-based agents can execute tasks over real codebases by combining planning and tool use [1]–[5]. In documentation automation, comparative studies analyze the quality of generated documentation and update workflows [6]–[8]. In testing and repair, LLM-assisted test generation, benchmarks, and reliability evaluations are prominent [9]–[12]. Approaches such as ReAct, Chain-of-Thought, and Tree-of-Thoughts provide foundations for more stable multi-step planning and action generation [13]–[15].

Unlike many task-specific approaches, AKIS aims to unify documentation, testing, and prototyping under a single orchestration core and a standardized integration layer. The platform-level design operationalizes plan–execute–reflect–validate reliability steps and persists evidence (plans, traces, artifacts) to support auditable execution and controlled operation.

3. PROJECT REQUIREMENTS

3.1. Functional Requirements

Code	Description
FR1	The system shall provide multi-step user sign-up and login flows.
FR2	The user shall be able to create and save agent configuration via the UI.
FR3	The user shall be able to start an agent job and monitor its status.
FR4	The Scribe agent shall analyze repository changes and generate documentation update artifacts.
FR5	The Trace agent shall generate test scenarios and automation artifacts from requirements and acceptance criteria.
FR6	The Proto agent shall execute an analysis–design–prototype flow from requirements and produce artifacts.
FR7	Access to external systems shall be standardized through MCP adapters.

3.2. Non-Functional Requirements

Code	Description
NFR1	The system shall be designed to run on OCI Free Tier resources under constrained CPU/RAM.
NFR2	Authentication shall be implemented via a stateless JWT stored in an HTTP-only cookie.

NFR3	Runtime validation shall enforce input/output contracts and schema compliance.
NFR4	Execution steps and state transitions shall be persisted as auditable records.
NFR5	External integrations shall be implemented via an MCP adapter layer (no direct vendor SDK coupling).
NFR6	The system shall handle failures using structured error handling, timeouts, and retry strategies.

3.3. Design Constraints and Assumptions

Key constraints that drive the design decisions are listed below:

- Backend: Fastify + TypeScript; ORM: Drizzle; database: PostgreSQL.
- Frontend: React + Vite SPA; styling: Tailwind CSS.
- External integrations: must be implemented through MCP adapters.
- Session management: 7-day JWT in an HTTP-only cookie; passwords are hashed with bcrypt.
- Operational simplicity: application and PostgreSQL are expected to run on the same VM for the prototype deployment.

3.4. Cloud and Infrastructure Requirements

The target runtime environment is Oracle Cloud Infrastructure (OCI) Free Tier on an Arm-based VM instance (4 OCPU, 24 GB RAM). Within this scope, the application backend and database are expected to run on the same virtual machine. Limited compute and memory resources directly affect CPU and RAM budgets, cold-start time, and inter-service overhead, and therefore influence key architectural decisions.

To satisfy these requirements, Fastify (lightweight HTTP server) and Drizzle (minimal-overhead ORM) are selected to keep runtime consumption under control and reduce dependency complexity. The SPA frontend can be served as static assets to avoid an additional server-side

rendering layer. For deployment, the design supports containerized execution with clear health checks and configuration via environment variables.

3.5. Academic Requirements

From an academic perspective, the architecture must be justified using criteria such as traceability, auditability, and evidence-based evaluation. Agent-based software engineering approaches typically employ planning and execution loops, complemented with reflection/verification steps to improve output quality and controllability [1], [3], [13]. The AKIS architecture adopts a plan–execute–reflect–validate loop to increase output consistency and enable reviewable results.

In addition, the prototype is expected to be deployable, runnable, and observable within the scope of Graduation Project I. Therefore, the implementation is organized as a modular monolith to keep the system understandable and maintainable while enabling iterative development and academic evaluation through logged evidence (plans, traces, artifacts).

4. SYSTEM DESIGN

4.1. High-Level Architecture Overview

AKIS is designed as a modular monolith backend within a single repository and a decoupled SPA frontend. The backend is decomposed into an API layer, the core orchestration module (AgentOrchestrator), agent implementations, and service/adapter layers. The frontend provides onboarding flows, agent configuration, and job execution/monitoring screens.

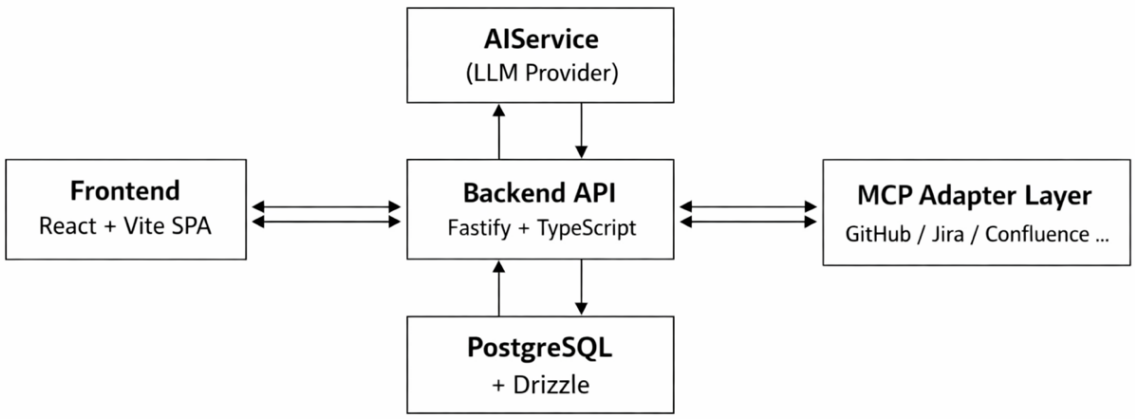

Figure 4.1: AKIS Platform high-level architectural components (backend–frontend–database–MCP integrations).

4.2. Backend Design

4.2.1. AgentOrchestrator and Agent Lifecycle

The central coordinator (AgentOrchestrator) runs all agents from a single control point without allowing agents to call each other directly. The orchestrator is responsible for job initiation, state management (pending → running → completed/failed), dependency injection (database, MCP adapters, AIService), error handling, and retry strategies when needed.

Agent workflows follow a four-step loop: planning (planner), execution (executor), reflection/improvement (reflection), and validation. This approach improves output quality while supporting controlled operation in steps that may require human approval.

During planning, the task is decomposed into sub-steps and a tool usage strategy is selected. During execution, the plan is executed step-by-step using injected tools such as MCP adapters, the HTTP client, and AIService. Reflection re-evaluates the output for consistency and requirement alignment and may request revisions. Validation checks the output against format and content contracts and stores job results as artifacts.

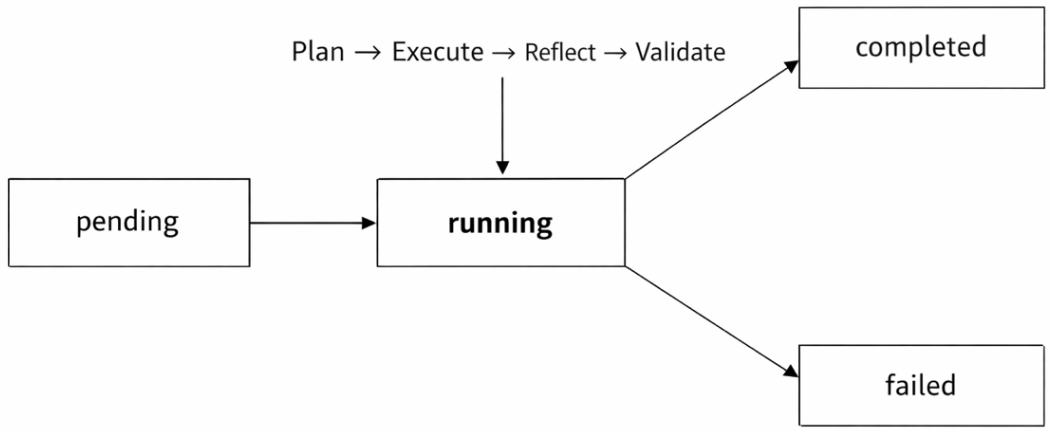

Figure 4.2: State machine (FSM) summary for the job lifecycle.

4.2.2. Authentication and Onboarding Flows

Authentication is implemented through email and password-based multi-step sign-up and login flows. Sessions are managed using a stateless JWT stored in an HTTP-only cookie. Email verification uses a 6-digit code with a 15-minute expiry; rate limiting and abuse controls are applied to the verification flow.

4.2.3. API Layer and Endpoint Summary

The API layer exposes REST endpoints for authentication, agent job management, and system health. Table 4.x summarizes core endpoints.

Method	Path	Purpose
GET	/health, /ready, /version	Health checks and version information.
POST	/auth/signup/start; /auth/signup/password; /auth/verify-email	Sign-up and email verification flows.
POST	/auth/login/start; /auth/login/complete; /auth/logout	Login and logout flows.
GET	/auth/me	Current user session information.
POST/GET	/api/agents/jobs	Create and list jobs.
GET	/api/agents/jobs/:id	Retrieve job details (state, plan, traces, artifacts).

4.2.4. Security Layer and Middleware

Security is built around HTTP-only session cookies, email verification codes, and rate limiting. Verification codes are time-limited and protected against brute-force attempts using attempt limits and backoff strategies. At the HTTP layer, CORS policies, security headers, and basic rate limiting reduce abuse risks. Sensitive values are intended to be masked in logs/traces, and error responses are designed to avoid leaking confidential details.

4.2.5. Module Structure and File Organization

The backend follows a modular monolith structure within a single repository. API routing is organized under src/api, while the orchestration kernel lives under src/core/orchestrator. Agent contracts, the state machine, and shared playbook structures are maintained under core modules. Access to external systems and LLM providers is isolated in service/adapter layers. This organization supports maintainable growth and clear separation of responsibilities.

4.3. Frontend Design

4.3.1. Information Architecture and Screens

The frontend is implemented as a React + Vite SPA and includes the following key screens:

- Authentication wizard: multi-step sign-up, two-step login, and privacy consent (where applicable).
- Dashboard: agent selection, configuration panels, job creation, and job monitoring.
- Job detail view: state, trace events, generated artifacts, and revision/approval information.

4.4. Data Model Design

The core data entities are summarized below. Table 4.1 provides an overview of primary database tables and their purposes.

Table	Purpose
users	User account, verification, and preference fields.
email_verification_tokens	6-digit verification codes with expiry and usage state.
oauth_accounts	OAuth provider accounts and tokens (where applicable).
jobs	Primary record for agent jobs; state, type, and metadata.
job_plans	Planner output and structured plan steps.
job_traces	Execution trace events and timestamps.
job_artifacts	Generated files/outputs and their metadata.
agent_configs	Per-user agent configuration records.

4.4.1. Entities and Relational Model Overview

The data model is built around user identity/session management, persistent agent configurations, and auditable job execution records. Each job can be associated with plans, trace events, artifacts, and audit/approval metadata. This enables reproducibility, debugging, and review of agent actions after execution.

4.4.2. Job, Trace, and Artifact Lifecycle

The platform's execution model is centered on the concept of a job. A job starts with an agent type, configuration, and input parameters. During execution, plan steps, trace events, and artifacts are continuously persisted. This makes it possible to inspect which tools were called, in what order, and what outputs were produced. A revision chain can relate multiple executions of the same task with different revision notes.

4.4.3. Column-Level Summary (Selected Tables)

This subsection summarizes key columns of core tables based on the technical snapshot. Columns are chosen to support traceability, auditability, and controlled operation. For readability in the academic report, the tables below focus on the most critical fields.

Table 4.2. jobs — Selected Columns

Column	Type	Purpose
id	UUID	Primary key.
type	VARCHAR	Agent type (scribe/trace/proto).
state	ENUM	Job state (pending/running/completed/failed/awaiting_approval).
payload	JSONB	Input parameters.
result	JSONB	Execution output.
mcpGatewayUrl	VARCHAR	MCP Gateway used for the job (if any).
requiresApproval	BOOLEAN	Whether human approval is required before execution.
parentJobId	UUID	Parent job for the revision chain.
createdAt	TIMESTAMP	Creation time.
updatedAt	TIMESTAMP	Last update time.

Table 4.3. job_traces — Selected Columns

Column	Type	Purpose
id	UUID	Primary key.
jobId	UUID	Foreign key to jobs.

eventType	ENUM	Type of trace event (tool_call, ai_call, error, etc.).
title	VARCHAR	Event summary.
durationMs	INTEGER	Duration in milliseconds.
status	VARCHAR	success/failed/warning/info.
gatewayUrl	VARCHAR	Gateway URL for MCP events.
timestamp	TIMESTAMP	Event timestamp.
inputSummary	TEXT	Redacted input summary.
outputSummary	TEXT	Redacted output summary.

4.5. Integration Design

External integrations are standardized through an MCP adapter layer. This design consolidates protocol diversity behind a single contract, improves auditability of tool use, and reduces vendor coupling. As a result, adding a new integration typically requires implementing a new adapter rather than changing core agent logic.

4.5.1. MCP Gateway and Adapter Approach

The MCP Gateway provides a single entry point to external systems (e.g., GitHub, document stores, and task management tools) through adapters. During agent execution, AgentOrchestrator routes tool calls to the gateway; the gateway dispatches requests to the appropriate adapter and returns normalized responses. This isolates integration code from the orchestration core and enables cross-cutting concerns—such as timeouts, error classification, and trace production—to be handled consistently.

4.5.2. Container Components and Runtime Model

For local development and demo deployment, the prototype supports a containerized runtime model. A typical setup runs PostgreSQL (persistence), the backend API (Fastify), and the MCP Gateway as separate containers. The frontend can be served as static assets or run separately via the Vite development server.

This approach maintains dependency version compatibility, enables a reproducible demo environment, and provides a portable deployment model for OCI. Health endpoints and structured logging serve as a baseline for observability and operations.

4.5.3. Integration Security and Access Control

Integration keys and access tokens are managed through environment variables, and the system aims to mask secret values in logs/traces. Network access between components can be restricted to required ports only. Rate limiting and request validation reduce abuse risks. In multi-user scenarios, job data is protected via user-based authorization checks to ensure isolation.

4.6. Test Plan and Quality Strategy

The quality assurance approach is based on early testing and contract validation:

- Unit tests: isolated tests for orchestration flows, validations, and service-layer logic.
- Contract tests: schema validation for agent inputs/outputs.
- Integration tests: protocol compliance and failure scenarios for MCP adapters.
- End-to-end checks: critical user flows such as sign-up, login, job creation, and monitoring.

In addition to automation, the plan includes manual verification steps such as checking critical screen flows against acceptance criteria and reviewing sample agent outputs (Scribe/Trace) on a representative repository during demo preparation.

4.7. Deployment and Operations

Given the OCI Free Tier target and the focus on operational simplicity, the solution adopts a modular monolith approach to minimize runtime overhead. The application and PostgreSQL are expected to run on the same VM instance to reduce operational complexity. Health endpoints (/health, /ready) and structured logging provide baseline operational visibility.

5. TASKS ACCOMPLISHED

5.1. Completed Work and Intermediate Outputs

In alignment with the project management approach defined in the Project Scope and Requirements document, the following work items have been completed for the current stage:

- Established the modular monolith backend baseline (Fastify + TypeScript) and core API skeleton.
- Implemented the PostgreSQL + Drizzle data access layer and the core table set.
- Developed authentication flows (multi-step sign-up/login, email verification, session handling).
- Implemented the orchestration kernel (AgentOrchestrator), state management, and the baseline job lifecycle.
- Built a prototype-level Scribe configuration and dashboard flow to run and monitor jobs for demo scenarios; enabled baseline output generation and trace persistence.
- Prepared the integration framework for the MCP adapter layer and initial connectivity scenarios.

As part of the quality strategy, unit and integration test suites and checks for core user flows have been added to reduce regression risk. The project targets a broad test suite (130+ scenarios) as the system evolves, while keeping the demo environment stable.

5.2. Current Status Table

Table 5.1 summarizes project phases from 0.1 to 1.0 in a consistent format. Status values reflect completed work, demo readiness, and ongoing development.

Phase	Focus	Status
Phase 0.1	Repository skeleton and baseline project setup	Completed
Phase 0.2	Backend API baseline (Fastify) and health endpoints	Completed
Phase 0.3	Database schema and Drizzle access layer	Completed

Phase 0.4	Authentication and onboarding flows	Completed
Phase 0.5	AgentOrchestrator core and job lifecycle (FSM)	Completed
Phase 0.6	Scribe configuration dashboard and job monitoring (demo flow)	Demo-ready
Phase 1.0	End-to-end Scribe job (artifact generation and validation)	In progress

REFERENCES

[1] Suri, S., Das, S. N., Singi, K., Dey, K., Sharma, V. S., & Kaulgud, V. (2023). Software engineering using autonomous agents: Are we there yet? ASE 2023.

[2] Bouzenia, I., Devanbu, P., & Pradel, M. (2025). RepairAgent: An autonomous, LLM-based agent for program repair. ICSE 2025.

[3] He, J., Treude, C., & Lo, D. (2025). LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. ACM TOSEM, 34(5).

[4] Yang, J., et al. (2024). SWE-agent: Agent-computer interfaces enable automated software engineering. NeurIPS 2024.

[5] Li, G., et al. (2023). CAMEL: Communicative agents for "mind" exploration of large language model society. NeurIPS 2023.

[6] Khan, J. Y., & Uddin, G. (2023). Automatic code documentation generation using GPT-3. ASE 2022.

[7] Dvivedi, S. S., et al. (2024). A comparative analysis of large language models for code documentation generation. AIware 2024.

[8] Yang, D., et al. (2025). DocAgent: A multi-agent system for automated code documentation generation. ACL 2025 (System Demonstrations).

[9] Wang, J., et al. (2024). Software testing with large language models: Survey, landscape, and vision. IEEE TSE, 50(4), 911-936.

[10] Bhatia, S., et al. (2024). Unit test generation using generative AI: A comparative performance analysis of autogeneration tools. LLM4Code 2024.

[11] Tony, C., et al. (2023). LLMSecEval: A dataset of natural language prompts for security evaluations. MSR 2023.

[12] Mündler, N., et al. (2024). SWT-Bench: Testing and validating real-world bug-fixes with code agents. NeurIPS 2024.

[13] Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. ICLR 2023.

[14] Wang, X., et al. (2023). Self-consistency improves chain-of-thought reasoning in language models. ICLR 2023.

[15] Yao, S., et al. (2023). Tree of thoughts: Deliberate problem solving with large language models. NeurIPS 2023.